# Capítulo 07 - Agente Analista (Analyst Agent)

## 7.1. Introdução ao Agente Analista

O **Agente Analista** é o cérebro analítico do Projeto Quimera. Ele recebe URLs de perfis coletados pelo Agente Batedor e realiza uma **análise profunda e multidimensional** para determinar se o perfil é interessante para o usuário.

### 7.1.1. Responsabilidades do Agente Analista

1. **Coletar dados completos do perfil:**
   - Biografia
   - Foto de perfil
   - Postagens recentes (fotos e legendas)
   - Métricas sociais (seguidores, seguindo, engajamento)
2. **Análise Textual (NLP):**
   - Extrair interesses e hobbies da biografia
   - Identificar personalidade através de linguagem
   - Análise de sentimento
   - Extração de entidades (nomes, locais, organizações)
3. **Análise Visual (Computer Vision):**
   - Detectar faces e estimar idade/gênero
   - Identificar contextos (praia, festa, academia)
   - Analisar estilo estético
   - Detectar presença recorrente de parceiro
4. **Inferência de Status de Relacionamento:**
   - Analisar sinais de estar solteira/comprometida
   - Calcular confiança da inferência
5. **Pontuação Final:**
   - Calcular pontuação em 3 dimensões (interessante, bonita, solteira)
   - Calcular pontuação final ponderada
   - Classificar perfil como qualificado ou desqualificado

6. **Identificar Pontos de Afinidade:**
   - Interesses em comum
   - Hobbies em comum
   - Locais em comum
   - Conexões sociais em comum

### 7.1.2. Desafios Principais

**1. Dados Incompletos**

Nem todos os perfis têm biografia preenchida ou postagens públicas. O sistema precisa lidar com dados ausentes.

**2. Ambiguidade**

Linguagem humana é ambígua. "Amo viajar" pode significar viagens frequentes ou apenas um desejo.

**3. Performance**

Modelos de ML/AI são pesados. Processar 1000 perfis pode levar horas se não for otimizado.

**4. Privacidade**

Perfis privados não podem ser analisados sem seguir primeiro.

---

## 7.2. Arquitetura do Agente Analista

### 7.2.1. Componentes Principais

Plain Text

```
┌──────────────────────────────────────────────────────────────┐
│                AGENTE ANALISTA                                │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ┌──────────────────────────────────────────┐               │
│   │  1. COLETOR DE DADOS                     │               │
│   │  - Navega até perfil                     │               │
│   │  - Extrai biografia, foto, postagens     │               │
│   │  - Salva dados brutos                    │               │
│   └──────────────────────────────────────────┘               │
│                        │                                      │
│                        ▼                                      │
│   ┌──────────────────────────────────────────┐               │
│   │  2. ANALISADOR TEXTUAL (NLP)             │               │
```

```
│   │  - Tokenização                          │                │
│   │  - Named Entity Recognition              │                │
│   │  - Análise de Sentimento                 │                │
│   │  - Extração de Interesses                │                │
│   │  - Inferência de Personalidade           │                │
│   └──────────────────────────────────────────┘                │
│                       │                                        │
│                       ▼                                        │
│   ┌──────────────────────────────────────────┐                │
│   │  3. ANALISADOR VISUAL (CV)               │                │
│   │  - Detecção de Faces                     │                │
│   │  - Estimativa de Idade/Gênero            │                │
│   │  - Classificação de Contexto             │                │
│   │  - Análise de Estilo                     │                │
│   └──────────────────────────────────────────┘                │
│                       │                                        │
│                       ▼                                        │
│   ┌──────────────────────────────────────────┐                │
│   │  4. INFERIDOR DE RELACIONAMENTO          │                │
│   │  - Detecta sinais de parceiro            │                │
│   │  - Analisa linguagem de biografia        │                │
│   │  - Calcula confiança                     │                │
│   └──────────────────────────────────────────┘                │
│                       │                                        │
│                       ▼                                        │
│   ┌──────────────────────────────────────────┐                │
│   │  5. CALCULADOR DE PONTUAÇÃO              │                │
│   │  - Pontuação Interessante (0-10)         │                │
│   │  - Pontuação Bonita (0-10)               │                │
│   │  - Pontuação Solteira (0-10)             │                │
│   │  - Pontuação Final Ponderada             │                │
│   └──────────────────────────────────────────┘                │
│                       │                                        │
│                       ▼                                        │
│   ┌──────────────────────────────────────────┐                │
│   │  6. IDENTIFICADOR DE AFINIDADES          │                │
│   │  - Compara com preferências usuário      │                │
│   │  - Identifica pontos em comum            │                │
│   │  - Gera descrições de afinidades         │                │
│   └──────────────────────────────────────────┘                │
│                       │                                        │
│                       ▼                                        │
│   ┌──────────────────────────────────────────┐                │
│   │  7. GERADOR DE EMBEDDINGS                │                │
│   │  - Gera embedding da biografia           │                │
│   │  - Gera embedding do perfil completo     │                │
│   │  - Salva no ChromaDB                     │                │
│   └──────────────────────────────────────────┘                │
```

```
│                    │                                    │
│                    ▼                                    │
│  ┌──────────────────────────────────────┐              │
│  │  8. PERSISTÊNCIA                     │              │
│  │  - Salva análise no MongoDB          │              │
│  │  - Atualiza perfil no PostgreSQL     │              │
│  │  - Emite evento PerfilAnalisado      │              │
│  └──────────────────────────────────────┘              │
│                                                        │
└────────────────────────────────────────────────────────┘
```

## 7.2.2. Fluxo de Execução

Plain Text

```
INÍCIO
  │
  ├─> Escutar fila Redis: eventos:PerfilColetado
  │
  ├─> Para cada perfil_id recebido:
  │     │
  │     ├─> 1. COLETAR DADOS
  │     │     ├─> Navegar até perfil
  │     │     ├─> Verificar se perfil é público
  │     │     ├─> Extrair biografia
  │     │     ├─> Baixar foto de perfil
  │     │     ├─> Extrair últimas 9 postagens
  │     │     └─> Salvar dados brutos
  │     │
  │     ├─> 2. ANÁLISE TEXTUAL (NLP)
  │     │     ├─> Limpar e tokenizar biografia
  │     │     ├─> Extrair entidades (NER)
  │     │     ├─> Extrair interesses e hobbies
  │     │     ├─> Análise de sentimento
  │     │     ├─> Inferir traços de personalidade
  │     │     └─> Salvar resultados
  │     │
  │     ├─> 3. ANÁLISE VISUAL (Computer Vision)
  │     │     ├─> Detectar faces na foto de perfil
  │     │     ├─> Estimar idade e gênero
  │     │     ├─> Para cada postagem:
  │     │     │     ├─> Detectar faces
  │     │     │     ├─> Classificar contexto
  │     │     │     ├─> Identificar objetos
  │     │     │     └─> Analisar estilo
  │     │     ├─> Agregar análises
  │     │     └─> Salvar resultados
```

```
   |      |
   |      ├─> 4. INFERIR STATUS DE RELACIONAMENTO
   |      |      ├─> Analisar presença recorrente de pessoa
   |      |      ├─> Analisar linguagem da biografia
   |      |      ├─> Analisar hashtags e menções
   |      |      ├─> Calcular probabilidade solteira
   |      |      └─> Salvar resultado
   |      |
   |      ├─> 5. CALCULAR PONTUAÇÕES
   |      |      ├─> Pontuação Interessante (baseada em NLP)
   |      |      ├─> Pontuação Bonita (baseada em CV)
   |      |      ├─> Pontuação Solteira (baseada em inferência)
   |      |      ├─> Pontuação Final = soma ponderada
   |      |      └─> Classificar: qualificado se >= limiar
   |      |
   |      ├─> 6. IDENTIFICAR AFINIDADES
   |      |      ├─> Comparar interesses com usuário
   |      |      ├─> Comparar hobbies com usuário
   |      |      ├─> Comparar localização
   |      |      ├─> Gerar descrições de afinidades
   |      |      └─> Salvar no PostgreSQL
   |      |
   |      ├─> 7. GERAR EMBEDDINGS
   |      |      ├─> Gerar embedding da biografia
   |      |      ├─> Gerar embedding do perfil completo
   |      |      └─> Salvar no ChromaDB
   |      |
   |      ├─> 8. PERSISTIR RESULTADOS
   |      |      ├─> Salvar análise completa no MongoDB
   |      |      ├─> Atualizar perfil no PostgreSQL
   |      |      ├─> Emitir evento PerfilAnalisado
   |      |      └─> Se qualificado: emitir PerfilQualificado
   |      |
   |      └─> Aguardar próximo perfil
   |
FIM
```

---

# 7.3. Análise Textual (NLP)

## 7.3.1. O que é NLP?

**Natural Language Processing (NLP)** é a área da IA que ensina computadores a entender linguagem humana. No nosso caso, usamos NLP para extrair informações de biografias e legendas de postagens.

## 7.3.2. Pipeline de NLP

Plain Text

```
Texto Bruto
    |
    ├─> 1. Limpeza
    |      ├─> Remover emojis
    |      ├─> Remover URLs
    |      ├─> Normalizar espaços
    |      └─> Converter para minúsculas
    |
    ├─> 2. Tokenização
    |      └─> Dividir em palavras/tokens
    |
    ├─> 3. Remoção de Stopwords
    |      └─> Remover palavras comuns (a, o, de, para)
    |
    ├─> 4. Stemming/Lemmatization
    |      └─> Reduzir palavras à raiz (viajando → viajar)
    |
    ├─> 5. Named Entity Recognition (NER)
    |      └─> Identificar entidades (pessoas, locais, organizações)
    |
    ├─> 6. Análise de Sentimento
    |      └─> Classificar como positivo/neutro/negativo
    |
    └─> 7. Extração de Features
           └─> Extrair interesses, hobbies, personalidade
```

## 7.3.3. Limpeza de Texto

**Exemplo de texto bruto:**

Plain Text

```
"🌴 Apaixonada por viagens ✈️ | Yoga 🧘‍♀️ | Fotografia 📸 | Varginha-MG 📍 |
IG: @meuoutroperfil"
```

**Após limpeza:**

Plain Text

```
"apaixonada por viagens yoga fotografia varginha mg"
```

**Código de limpeza:**

Python

```python
import re

def limpar_texto(texto):
    """Remove emojis, URLs e caracteres especiais."""
    # Remover URLs
    texto = re.sub(r'http\S+|www\S+', '', texto)

    # Remover menções (@usuario)
    texto = re.sub(r'@\w+', '', texto)

    # Remover hashtags (#tag) mas manter o texto
    texto = re.sub(r'#(\w+)', r'\1', texto)

    # Remover emojis
    texto = re.sub(r'[^\w\s,.-]', '', texto)

    # Normalizar espaços
    texto = re.sub(r'\s+', ' ', texto).strip()

    # Converter para minúsculas
    texto = texto.lower()

    return texto

# Uso
biografia = "🌴 Apaixonada por viagens ✈️ | Yoga 🧘‍♀️"
biografia_limpa = limpar_texto(biografia)
print(biografia_limpa)  # "apaixonada por viagens yoga"
```

### 7.3.4. Tokenização

**Tokenização** é dividir o texto em palavras individuais (tokens).

Python

```python
from nltk.tokenize import word_tokenize
import nltk

# Baixar dados necessários (executar uma vez)
nltk.download('punkt')

def tokenizar(texto):
    """Divide texto em tokens."""
    return word_tokenize(texto, language='portuguese')
```

```python
# Uso
texto = "apaixonada por viagens yoga fotografia"
tokens = tokenizar(texto)
print(tokens)  # ['apaixonada', 'por', 'viagens', 'yoga', 'fotografia']
```

### 7.3.5. Remoção de Stopwords

**Stopwords** são palavras comuns que não carregam significado (a, o, de, para, etc.).

Python

```python
from nltk.corpus import stopwords
import nltk

# Baixar stopwords (executar uma vez)
nltk.download('stopwords')

def remover_stopwords(tokens):
    """Remove stopwords de lista de tokens."""
    stop_words = set(stopwords.words('portuguese'))
    return [token for token in tokens if token not in stop_words]

# Uso
tokens = ['apaixonada', 'por', 'viagens', 'yoga', 'fotografia']
tokens_filtrados = remover_stopwords(tokens)
print(tokens_filtrados)  # ['apaixonada', 'viagens', 'yoga', 'fotografia']
```

### 7.3.6. Named Entity Recognition (NER)

**NER** identifica entidades nomeadas no texto: pessoas, locais, organizações, datas.

Python

```python
from transformers import AutoTokenizer, AutoModelForTokenClassification
from transformers import pipeline

# Carregar modelo BERT para português
modelo = "neuralmind/bert-base-portuguese-cased"
tokenizer = AutoTokenizer.from_pretrained(modelo)
model = AutoModelForTokenClassification.from_pretrained(modelo)

# Criar pipeline de NER
ner_pipeline = pipeline("ner", model=model, tokenizer=tokenizer,
aggregation_strategy="simple")
```

```python
def extrair_entidades(texto):
    """Extrai entidades nomeadas do texto."""
    entidades = ner_pipeline(texto)
    return entidades

# Uso
texto = "Moro em Varginha e trabalho na Universidade Federal"
entidades = extrair_entidades(texto)
print(entidades)
# [
#   {'entity_group': 'LOC', 'word': 'Varginha', 'score': 0.95},
#   {'entity_group': 'ORG', 'word': 'Universidade Federal', 'score': 0.89}
# ]
```

### 7.3.7. Análise de Sentimento

**Análise de Sentimento** classifica o texto como positivo, neutro ou negativo.

Python

```python
from transformers import pipeline

# Carregar modelo de sentimento para português
sentiment_pipeline = pipeline(
    "sentiment-analysis",
    model="cardiffnlp/twitter-xlm-roberta-base-sentiment"
)

def analisar_sentimento(texto):
    """Analisa sentimento do texto."""
    resultado = sentiment_pipeline(texto)[0]
    return {
        'sentimento': resultado['label'],  # 'positive', 'neutral',
'negative'
        'confianca': resultado['score']
    }

# Uso
texto = "Amo viajar e conhecer lugares novos!"
sentimento = analisar_sentimento(texto)
print(sentimento)  # {'sentimento': 'positive', 'confianca': 0.98}
```

### 7.3.8. Extração de Interesses e Hobbies

**Estratégia:** Usar um dicionário de palavras-chave associadas a interesses.

Python

```python
# Dicionário de interesses
DICIONARIO_INTERESSES = {
    'viagens': ['viagem', 'viajar', 'turismo', 'mochileiro', 'aventura'],
    'fotografia': ['fotografia', 'fotografar', 'foto', 'camera',
'fotografo'],
    'musica': ['musica', 'cantar', 'tocar', 'violao', 'guitarra', 'piano'],
    'esportes': ['esporte', 'correr', 'academia', 'treino', 'fitness',
'yoga'],
    'leitura': ['ler', 'livro', 'leitura', 'literatura'],
    'gastronomia': ['cozinhar', 'culinaria', 'gastronomia', 'chef',
'receita'],
    'arte': ['arte', 'pintura', 'desenho', 'artista'],
    'tecnologia': ['tecnologia', 'programacao', 'codigo', 'desenvolvedor'],
}

def extrair_interesses(texto):
    """Extrai interesses do texto baseado em palavras-chave."""
    texto_limpo = limpar_texto(texto)
    tokens = tokenizar(texto_limpo)

    interesses_encontrados = []

    for interesse, palavras_chave in DICIONARIO_INTERESSES.items():
        for token in tokens:
            if token in palavras_chave:
                interesses_encontrados.append(interesse)
                break  # Encontrou, passar para próximo interesse

    return interesses_encontrados

# Uso
biografia = "Apaixonada por viagens, fotografia e yoga"
interesses = extrair_interesses(biografia)
print(interesses)  # ['viagens', 'fotografia', 'esportes']
```

### 7.3.9. Inferência de Personalidade (Big Five)

**Big Five** é um modelo psicológico que descreve personalidade em 5 dimensões:

1. **Extroversão:** Sociável vs. Reservado
2. **Abertura:** Criativo vs. Convencional
3. **Conscienciosidade:** Organizado vs. Espontâneo
4. **Amabilidade:** Compassivo vs. Competitivo
5. **Neuroticismo:** Ansioso vs. Estável

**Estratégia:** Usar palavras-chave para inferir traços.

Python

```python
# Dicionário de traços de personalidade
TRACOS_PERSONALIDADE = {
    'extroversao': {
        'alto': ['festa', 'social', 'amigos', 'conhecer', 'conversar'],
        'baixo': ['introvertido', 'quieto', 'sozinho', 'casa']
    },
    'abertura': {
        'alto': ['criativo', 'arte', 'novo', 'aventura', 'experimentar'],
        'baixo': ['tradicional', 'rotina', 'conservador']
    },
    'conscienciosidade': {
        'alto': ['organizado', 'planejamento', 'disciplina', 'metas'],
        'baixo': ['espontaneo', 'flexivel', 'improviso']
    },
    'amabilidade': {
        'alto': ['ajudar', 'caridade', 'empatia', 'gentil', 'amor'],
        'baixo': ['competitivo', 'assertivo', 'direto']
    },
    'neuroticismo': {
        'alto': ['ansioso', 'estresse', 'preocupado', 'nervoso'],
        'baixo': ['calmo', 'relaxado', 'tranquilo', 'paz']
    }
}

def inferir_personalidade(texto):
    """Infere traços de personalidade do texto."""
    texto_limpo = limpar_texto(texto)
    tokens = tokenizar(texto_limpo)

    pontuacoes = {}

    for traco, niveis in TRACOS_PERSONALIDADE.items():
        pontuacao_alto = sum(1 for token in tokens if token in
niveis['alto'])
        pontuacao_baixo = sum(1 for token in tokens if token in
niveis['baixo'])

        # Normalizar para 0-1
        total = pontuacao_alto + pontuacao_baixo
        if total > 0:
            pontuacoes[traco] = pontuacao_alto / total
        else:
            pontuacoes[traco] = 0.5  # Neutro
```

```python
    return pontuacoes

# Uso
biografia = "Amo conhecer pessoas novas, festas e aventuras criativas!"
personalidade = inferir_personalidade(biografia)
print(personalidade)
# {
#   'extroversao': 0.75,
#   'abertura': 0.80,
#   'conscienciosidade': 0.50,
#   'amabilidade': 0.50,
#   'neuroticismo': 0.50
# }
```

---

# 7.4. Análise Visual (Computer Vision)

## 7.4.1. O que é Computer Vision?

**Computer Vision (CV)** é a área da IA que ensina computadores a "ver" e interpretar imagens. No nosso caso, usamos CV para analisar fotos de perfil e postagens.

## 7.4.2. Pipeline de Computer Vision

Plain Text

```
Imagem
    |
    ├─> 1. Pré-processamento
    |     ├─> Redimensionar
    |     ├─> Normalizar cores
    |     └─> Converter para tensor
    |
    ├─> 2. Detecção de Faces
    |     ├─> Localizar faces (bounding boxes)
    |     └─> Extrair região da face
    |
    ├─> 3. Análise de Face
    |     ├─> Estimar idade
    |     ├─> Estimar gênero
    |     ├─> Detectar emoção
    |     └─> Analisar atributos (óculos, barba, etc.)
    |
    ├─> 4. Classificação de Contexto
    |     └─> Identificar cenário (praia, festa, academia)
    |
```

```
    ├─> 5. Detecção de Objetos
    │      └─> Identificar objetos na imagem
    │
    └─> 6. Análise de Estilo
           └─> Classificar estilo estético
```

### 7.4.3. Detecção de Faces

**Biblioteca:** OpenCV com Haar Cascades ou MTCNN (mais preciso).

Python

```python
import cv2
from mtcnn import MTCNN

# Inicializar detector
detector = MTCNN()

def detectar_faces(caminho_imagem):
    """Detecta faces em uma imagem."""
    # Carregar imagem
    imagem = cv2.imread(caminho_imagem)
    imagem_rgb = cv2.cvtColor(imagem, cv2.COLOR_BGR2RGB)

    # Detectar faces
    faces = detector.detect_faces(imagem_rgb)

    resultados = []
    for face in faces:
        x, y, w, h = face['box']
        confianca = face['confidence']

        resultados.append({
            'bbox': [x, y, w, h],
            'confianca': confianca
        })

    return resultados

# Uso
faces = detectar_faces("foto_perfil.jpg")
print(f"Encontradas {len(faces)} faces")
```

### 7.4.4. Estimativa de Idade e Gênero

**Modelo:** DeepFace (biblioteca que usa modelos pré-treinados).

Python

```python
from deepface import DeepFace

def analisar_face(caminho_imagem):
    """Analisa idade, gênero e emoção de uma face."""
    try:
        analise = DeepFace.analyze(
            caminho_imagem,
            actions=['age', 'gender', 'emotion'],
            enforce_detection=False
        )

        return {
            'idade': analise[0]['age'],
            'genero': analise[0]['dominant_gender'],
            'emocao': analise[0]['dominant_emotion'],
            'confianca_genero': analise[0]['gender'][analise[0]
['dominant_gender']]
        }
    except Exception as e:
        print(f"Erro ao analisar face: {e}")
        return None

# Uso
resultado = analisar_face("foto_perfil.jpg")
print(resultado)
# {
#   'idade': 28,
#   'genero': 'Woman',
#   'emocao': 'happy',
#   'confianca_genero': 0.92
# }
```

### 7.4.5. Classificação de Contexto

**Modelo:** CLIP (Contrastive Language-Image Pre-training) da OpenAI.

CLIP pode classificar imagens usando descrições em linguagem natural.

Python

```python
from transformers import CLIPProcessor, CLIPModel
from PIL import Image

# Carregar modelo CLIP
modelo = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
```

```python
# Contextos possíveis
CONTEXTOS = [
    "praia",
    "festa",
    "academia",
    "restaurante",
    "viagem",
    "casa",
    "trabalho",
    "natureza",
    "cidade"
]

def classificar_contexto(caminho_imagem):
    """Classifica o contexto de uma imagem."""
    # Carregar imagem
    imagem = Image.open(caminho_imagem)

    # Preparar inputs
    inputs = processor(
        text=CONTEXTOS,
        images=imagem,
        return_tensors="pt",
        padding=True
    )

    # Obter probabilidades
    outputs = modelo(**inputs)
    logits_per_image = outputs.logits_per_image
    probs = logits_per_image.softmax(dim=1)

    # Obter contexto com maior probabilidade
    idx_max = probs.argmax().item()
    contexto = CONTEXTOS[idx_max]
    confianca = probs[0][idx_max].item()

    return {
        'contexto': contexto,
        'confianca': confianca
    }

# Uso
resultado = classificar_contexto("postagem1.jpg")
print(resultado)  # {'contexto': 'praia', 'confianca': 0.87}
```

### 7.4.6. Detecção de Objetos

**Modelo:** YOLO (You Only Look Once) ou Detectron2.

Python

```python
from transformers import DetrImageProcessor, DetrForObjectDetection
from PIL import Image
import torch

# Carregar modelo
processor = DetrImageProcessor.from_pretrained("facebook/detr-resnet-50")
modelo = DetrForObjectDetection.from_pretrained("facebook/detr-resnet-50")

def detectar_objetos(caminho_imagem, threshold=0.9):
    """Detecta objetos em uma imagem."""
    # Carregar imagem
    imagem = Image.open(caminho_imagem)

    # Preparar inputs
    inputs = processor(images=imagem, return_tensors="pt")

    # Inferência
    outputs = modelo(**inputs)

    # Processar resultados
    target_sizes = torch.tensor([imagem.size[::-1]])
    results = processor.post_process_object_detection(
        outputs,
        target_sizes=target_sizes,
        threshold=threshold
    )[0]

    objetos = []
    for score, label, box in zip(results["scores"], results["labels"],
results["boxes"]):
        objetos.append({
            'objeto': modelo.config.id2label[label.item()],
            'confianca': score.item(),
            'bbox': box.tolist()
        })

    return objetos

# Uso
objetos = detectar_objetos("postagem1.jpg")
print(objetos)
# [
#   {'objeto': 'person', 'confianca': 0.98, 'bbox': [100, 100, 300, 400]},
```

```
#   {'objeto': 'beach', 'confianca': 0.92, 'bbox': [0, 0, 640, 480]}
# ]
```

---

## 7.5. Inferência de Status de Relacionamento

### 7.5.1. Estratégia de Inferência

Para inferir se uma pessoa está solteira ou comprometida, analisamos:

1. **Presença recorrente de pessoa nas fotos**
2. **Linguagem da biografia** (menções a parceiro)
3. **Hashtags e menções** (#namorado, #marido, @parceiro)
4. **Contexto das fotos** (fotos românticas)

### 7.5.2. Detecção de Pessoa Recorrente

Python

```python
def detectar_pessoa_recorrente(postagens, threshold=0.5):
    """
    Detecta se há uma pessoa recorrente nas postagens.

    Args:
        postagens: Lista de análises de postagens
        threshold: Percentual mínimo para considerar recorrente

    Returns:
        bool: True se há pessoa recorrente
    """
    total_postagens = len(postagens)
    if total_postagens == 0:
        return False

    # Contar postagens com 2+ pessoas
    postagens_com_dupla = 0
    for postagem in postagens:
        if postagem.get('faces_detectadas', 0) >= 2:
            postagens_com_dupla += 1

    # Se mais de 50% das postagens têm 2+ pessoas, provável relacionamento
    proporcao = postagens_com_dupla / total_postagens
    return proporcao >= threshold

# Uso
```

```python
postagens = [
    {'faces_detectadas': 2},
    {'faces_detectadas': 2},
    {'faces_detectadas': 1},
]
tem_pessoa_recorrente = detectar_pessoa_recorrente(postagens)
print(tem_pessoa_recorrente)  # True
```

### 7.5.3. Análise de Linguagem

Python

```python
# Palavras-chave que indicam relacionamento
PALAVRAS_RELACIONAMENTO = [
    'namorado', 'namorada', 'marido', 'esposa', 'noivo', 'noiva',
    'amor', 'mozao', 'mozão', 'bae', 'amor da minha vida',
    'casado', 'casada', 'comprometido', 'comprometida'
]

def analisar_linguagem_relacionamento(texto):
    """Analisa se o texto indica relacionamento."""
    texto_limpo = limpar_texto(texto)

    for palavra in PALAVRAS_RELACIONAMENTO:
        if palavra in texto_limpo:
            return True

    return False

# Uso
biografia = "Apaixonada por viagens com meu namorado"
tem_relacionamento = analisar_linguagem_relacionamento(biografia)
print(tem_relacionamento)  # True
```

### 7.5.4. Cálculo de Probabilidade

Python

```python
def calcular_probabilidade_solteira(analise_perfil):
    """
    Calcula probabilidade de estar solteira.

    Returns:
        float: Probabilidade entre 0 e 1
    """
    # Começar com probabilidade neutra
```

```python
    prob_solteira = 0.5

    # Evidências contra estar solteira (reduzem probabilidade)
    if analise_perfil.get('pessoa_recorrente'):
        prob_solteira -= 0.4

    if analise_perfil.get('linguagem_relacionamento'):
        prob_solteira -= 0.3

    # Evidências a favor de estar solteira (aumentam probabilidade)
    if analise_perfil.get('hashtags_solteira'):  # #solteira, #single
        prob_solteira += 0.3

    # Limitar entre 0 e 1
    prob_solteira = max(0.0, min(1.0, prob_solteira))

    return prob_solteira

# Uso
analise = {
    'pessoa_recorrente': False,
    'linguagem_relacionamento': False,
    'hashtags_solteira': True
}
prob = calcular_probabilidade_solteira(analise)
print(f"Probabilidade de estar solteira: {prob:.2f}")  # 0.80
```

---

## 7.6. Cálculo de Pontuações

### 7.6.1. Pontuação Interessante (0-10)

Baseada em:

- Interesses em comum
- Personalidade compatível
- Riqueza da biografia

Python

```python
def calcular_pontuacao_interessante(analise_nlp, preferencias_usuario):
    """Calcula pontuação de quão interessante é o perfil."""
    pontuacao = 0.0

    # Interesses em comum (até 5 pontos)
    interesses_perfil = set(analise_nlp.get('interesses', []))
```

```
    interesses_usuario =
set(preferencias_usuario.get('interesses_desejados', []))
    interesses_comuns = interesses_perfil.intersection(interesses_usuario)
    pontuacao += min(len(interesses_comuns) * 1.5, 5.0)

    # Personalidade compatível (até 3 pontos)
    personalidade = analise_nlp.get('personalidade', {})
    if personalidade.get('extroversao', 0) > 0.6:
        pontuacao += 1.5
    if personalidade.get('abertura', 0) > 0.6:
        pontuacao += 1.5

    # Biografia rica (até 2 pontos)
    biografia = analise_nlp.get('biografia_original', '')
    if len(biografia) > 50:
        pontuacao += 1.0
    if len(biografia) > 100:
        pontuacao += 1.0

    return min(pontuacao, 10.0)
```

### 7.6.2. Pontuação Bonita (0-10)

Baseada em:

- Qualidade das fotos
- Estilo estético
- Contextos das fotos

Python

```
def calcular_pontuacao_bonita(analise_cv):
    """Calcula pontuação estética do perfil."""
    pontuacao = 5.0  # Começar neutro

    # Qualidade da foto de perfil (até +3)
    qualidade_foto = analise_cv.get('foto_perfil', {}).get('qualidade', 0.5)
    pontuacao += qualidade_foto * 3

    # Estilo das fotos (até +2)
    estilo = analise_cv.get('analise_agregada',
{}).get('estilo_predominante')
    if estilo in ['elegante', 'moderno']:
        pontuacao += 2.0
    elif estilo in ['casual', 'esportivo']:
        pontuacao += 1.0
```

```
    # Contextos interessantes (até +2)
    contextos = analise_cv.get('analise_agregada',
{}).get('contextos_frequentes', [])
    if 'viagem' in contextos:
        pontuacao += 1.0
    if 'festa' in contextos or 'restaurante' in contextos:
        pontuacao += 1.0

    return min(pontuacao, 10.0)
```

### 7.6.3. Pontuação Solteira (0-10)

Baseada na probabilidade calculada anteriormente.

Python

```
def calcular_pontuacao_solteira(probabilidade_solteira):
    """Converte probabilidade (0-1) em pontuação (0-10)."""
    return probabilidade_solteira * 10
```

### 7.6.4. Pontuação Final Ponderada

Python

```
def calcular_pontuacao_final(pontuacoes, pesos):
    """
    Calcula pontuação final ponderada.

    Args:
        pontuacoes: Dict com 'interessante', 'bonita', 'solteira'
        pesos: Dict com pesos para cada dimensão (soma = 1.0)

    Returns:
        float: Pontuação final (0-10)
    """
    pontuacao_final = (
        pontuacoes['interessante'] * pesos['peso_interessante'] +
        pontuacoes['bonita'] * pesos['peso_bonita'] +
        pontuacoes['solteira'] * pesos['peso_solteira']
    )

    return round(pontuacao_final, 1)

# Uso
pontuacoes = {
    'interessante': 8.5,
```

```python
    'bonita': 7.8,
    'solteira': 9.0
}

pesos = {
    'peso_interessante': 0.40,
    'peso_bonita': 0.30,
    'peso_solteira': 0.30
}

pontuacao_final = calcular_pontuacao_final(pontuacoes, pesos)
print(f"Pontuação final: {pontuacao_final}")  # 8.5
```

---

# 7.7. Geração de Embeddings

## 7.7.1. O que são Embeddings?

**Embeddings** são representações numéricas de texto em um espaço vetorial de alta dimensão. Textos similares têm embeddings próximos.

**Exemplo:**

Plain Text

```
"Amo viajar" → [0.123, -0.456, 0.789, ...]  (384 dimensões)
"Adoro viagens" → [0.125, -0.450, 0.785, ...]  (próximo!)
"Gosto de programar" → [-0.234, 0.567, -0.123, ...]  (distante)
```

## 7.7.2. Gerando Embeddings

Python

```python
from sentence_transformers import SentenceTransformer

# Carregar modelo
modelo_embedding = SentenceTransformer('paraphrase-multilingual-mpnet-base-v2')

def gerar_embedding(texto):
    """Gera embedding de um texto."""
    embedding = modelo_embedding.encode(texto)
    return embedding.tolist()

# Uso
```

```
biografia = "Apaixonada por viagens, fotografia e yoga"
embedding = gerar_embedding(biografia)
print(f"Dimensões: {len(embedding)}")  # 768
print(f"Primeiros 5 valores: {embedding[:5]}")
```

### 7.7.3. Salvando no ChromaDB

Python

```python
import chromadb

# Conectar ao ChromaDB
client = chromadb.HttpClient(host='chromadb', port=8000)

# Obter coleção
collection = client.get_or_create_collection(name="perfis_embeddings")

def salvar_embedding(perfil_id, biografia, embedding, pontuacao_final):
    """Salva embedding no ChromaDB."""
    collection.add(
        embeddings=[embedding],
        documents=[biografia],
        metadatas=[{
            'perfil_id': perfil_id,
            'pontuacao_final': pontuacao_final
        }],
        ids=[perfil_id]
    )

# Uso
salvar_embedding(
    perfil_id="abc123",
    biografia="Apaixonada por viagens...",
    embedding=embedding,
    pontuacao_final=8.5
)
```

### 7.7.4. Busca por Similaridade

Python

```python
def buscar_perfis_similares(biografia_referencia, n_resultados=10):
    """Busca perfis similares baseado na biografia."""
    # Gerar embedding da biografia de referência
    embedding_referencia = gerar_embedding(biografia_referencia)
```

```python
    # Buscar no ChromaDB
    resultados = collection.query(
        query_embeddings=[embedding_referencia],
        n_results=n_resultados,
        where={"pontuacao_final": {"$gte": 7.5}}  # Apenas qualificados
    )

    return resultados

# Uso
biografia_ref = "Amo viajar e conhecer culturas diferentes"
similares = buscar_perfis_similares(biografia_ref)
print(f"Encontrados {len(similares['ids'][0])} perfis similares")
```

---

# 7.8. Otimizações de Performance

## 7.8.1. Batch Processing

Processar múltiplos perfis em lote é mais eficiente que um por vez.

Python

```python
def processar_batch(perfis_ids, batch_size=32):
    """Processa perfis em lotes."""
    for i in range(0, len(perfis_ids), batch_size):
        batch = perfis_ids[i:i+batch_size]

        # Processar batch
        for perfil_id in batch:
            processar_perfil(perfil_id)
```

## 7.8.2. Caching de Modelos

Carregar modelos ML é lento. Cache-os em memória.

Python

```python
class CacheModelos:
    _instancia = None

    def __new__(cls):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
            cls._instancia._carregar_modelos()
        return cls._instancia
```

```
    def _carregar_modelos(self):
        """Carrega todos os modelos uma vez."""
        print("Carregando modelos...")
        self.modelo_ner = pipeline("ner", model="neuralmind/bert-base-
portuguese-cased")
        self.modelo_sentimento = pipeline("sentiment-analysis",
model="cardiffnlp/twitter-xlm-roberta-base-sentiment")
        self.modelo_embedding = SentenceTransformer('paraphrase-multilingual-
mpnet-base-v2')
        print("Modelos carregados!")

# Uso (singleton)
cache = CacheModelos()
ner_pipeline = cache.modelo_ner
```

### 7.8.3. Processamento Assíncrono

Use asyncio para I/O bound operations.

Python

```
import asyncio
import aiohttp

async def baixar_imagem_async(url):
    """Baixa imagem de forma assíncrona."""
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as response:
            return await response.read()

async def processar_perfil_async(perfil_id):
    """Processa perfil de forma assíncrona."""
    # Baixar imagens em paralelo
    urls_imagens = obter_urls_imagens(perfil_id)
    tasks = [baixar_imagem_async(url) for url in urls_imagens]
    imagens = await asyncio.gather(*tasks)

    # Processar imagens
    for imagem in imagens:
        analisar_imagem(imagem)
```

---

## 7.9. Conclusão do Capítulo

Este capítulo cobriu o **Agente Analista** em profundidade:

- **NLP:** Limpeza, tokenização, NER, sentimento, extração de interesses, personalidade
- **Computer Vision:** Detecção de faces, estimativa de idade/gênero, classificação de contexto, detecção de objetos
- **Inferência de Relacionamento:** Detecção de pessoa recorrente, análise de linguagem, cálculo de probabilidade
- **Pontuações:** Interessante, Bonita, Solteira, Final Ponderada
- **Embeddings:** Geração, armazenamento no ChromaDB, busca por similaridade
- **Otimizações:** Batch processing, caching, processamento assíncrono

No próximo capítulo, veremos o **Motor de Persuasão**, que define estratégias de comunicação baseadas em psicologia.

**Próximo Capítulo:** Capítulo 08 - Motor de Persuasão

---

**Referências:**


- Transformers (Hugging Face): https://huggingface.co/docs/transformers/
- Sentence Transformers: https://www.sbert.net/
- DeepFace: https://github.com/serengil/deepface
- CLIP (OpenAI): https://github.com/openai/CLIP
- ChromaDB: https://docs.trychroma.com/